Refactoring

- Refactoring es una **transformación** que preserva el comportamiento, pero mejora el diseño

Bill Opdyke, PhD Thesis "Refactoring Object-Oriented Frameworks". Univ. of Illinois at Urbana-Champaign (UIUC). 1992. Director: Ralph Johnson.

Veamos un ejemplo….

- Retornar un string formateado para mostrar con los puntajes de los partido de tenis de una fecha específica. Para cada jugador se muestra el nombre, los games ganados en cada set y los puntos que el partido le permitió ganar.

Puntajes para los partidos de la fecha 7/3/2026

Partido:

Puntaje del jugador: Tomás Etcheverry: 6; 5; 7; Puntos del partido: 36

Puntaje del jugador: Thiago Tirante: 4; 7; 6; Puntos del partido: 34

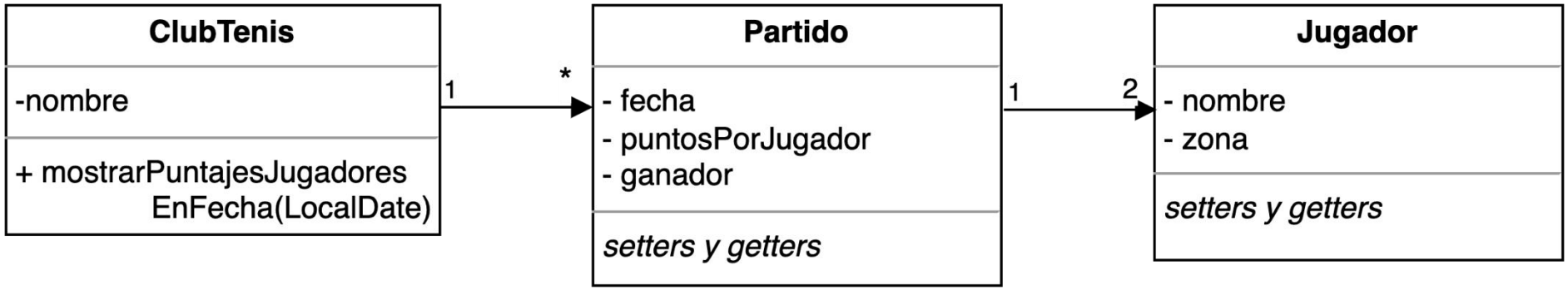
Partido:

.....

Código actual

```java
public class ClubTenis {
 private String nombre;
 private List<Partido> coleccionPartidos;
 public String mostrarPuntajesJugadoresEnFecha(LocalDate fecha) {
  int totalGames = 0;
  List<Partido> partidosFecha;
  String result = "Puntajes para los partidos de la fecha " + fecha.toString() + "\n";
  partidosFecha = coleccionPartidos.stream().filter(p -> p.fecha().equals(fecha)).collect(Collectors.toList());
  for (Partido p : partidosFecha) {
		totalGames = 0;
		Jugador j1 = p.jugador1();
		result += "Partido: " + "\n";
		result += "Puntaje del jugador: " + j1.nombre() + ": ";
		for (int gamesGanados : p.puntosPorSetDe(j1)) {
			result += Integer.toString(gamesGanados) + ";";
			totalGames += gamesGanados;
		}
		result += "Puntos del partido: ";
		if (j1.zona() == "A") result += Integer.toString(totalGames * 2);
		if (j1.zona() == "B") result += Integer.toString(totalGames);
		if (j1.zona() == "C")
			if (p.ganador() == j1)
				result += Integer.toString(totalGames);
			else
				result += Integer.toString(0);
		Jugador j2 = p.jugador2();
		totalGames = 0;
		result += "Puntaje del jugador: " + j2.nombre() + ": ";
		for (int gamesGanados : p.puntosPorSetDe(j2)) {
			result += Integer.toString(gamesGanados) + ";";
			totalGames += gamesGanados;
		}
		result += "Puntos del partido: ";
		if (j2.zona() == "A") result += Integer.toString(totalGames * 2);
		if (j2.zona() == "B") result += Integer.toString(totalGames);
		if (j2.zona() == "C")
			if (p.ganador() == j2)
				result += Integer.toString(totalGames);
			else
				result += Integer.toString(0);
  }
  return result;
}
```

Diagrama inicial



Cambios pedidos …

- Existen tests de unidad que pasan correctamente con el código actual.
- Pero... se vienen cambios de funcionalidad que serían difíciles de acomodar con el código actual
  - Cambiará la manera de calcular los puntos
  - Pueden aparecer nuevas zonas
  - Puede necesitarse el resultado en otro formato
- Es conveniente refactorizar el código actual para que luego incorporar los cambios que aparezcan sea más rápido y fácil

Ejemplo del club de tenis

- ¿Por dónde empezamos?
- ¿Cuáles son los problemas que tiene el código, que atentan contra su extensibilidad y reuso?

```java
public class ClubTenis {
 private String nombre;
 private List<Partido> coleccionPartidos;
 public String mostrarPuntajesJugadoresEnFecha(LocalDate fecha) {
  int totalGames = 0;
  List<Partido> partidosFecha;
  String result = "Puntajes para los partidos de la fecha " + fecha.toString() + "\n";
  partidosFecha = coleccionPartidos.stream().filter(p -> p.fecha().equals(fecha)).collect(Collectors.toList());
  for (Partido p : partidosFecha) {
        totalGames = 0;
        Jugador j1 = p.jugador1();
        result += "Partido: " + "\n";
        result += "Puntaje del jugador: " + j1.nombre() + ": ";
        for (int gamesGanados : p.puntosPorSetDe(j1)) {
              result += Integer.toString(gamesGanados) + ";";
              totalGames += gamesGanados;
        }
        result += "Puntos del partido: ";
        if (j1.zona() == "A") result += Integer.toString(totalGames * 2);
        if (j1.zona() == "B") result += Integer.toString(totalGames);
        if (j1.zona() == "C")
              if (p.ganador() == j1)
                    result += Integer.toString(totalGames);
              else
                    result += Integer.toString(0);
        Jugador j2 = p.jugador2();
        totalGames = 0;
        result += "Puntaje del jugador: " + j2.nombre() + ": ";
        for (int gamesGanados : p.puntosPorSetDe(j2)) {
              result += Integer.toString(gamesGanados) + ";";
              totalGames += gamesGanados;
        }
        result += "Puntos del partido: ";
        if (j2.zona() == "A") result += Integer.toString(totalGames * 2);
        if (j2.zona() == "B") result += Integer.toString(totalGames);
        if (j2.zona() == "C")
              if (p.ganador() == j2)
                    result += Integer.toString(totalGames);
              else
                    result += Integer.toString(0);
  }
  return result;}
```

MÉTODO LARGO!

Refactoring Extract Method

- Motivación :
  - Métodos largos
  - Métodos muy comentados
  - Incrementar reuso
  - Incrementar legibilidad

Refactoring: Extract Method

- **Precondiciones**:
  1. La porción de código a extraer deber contener una unidad sintáctica completa (una expresión, sentencia, estructura de control, lista de sentencias con todas sus partes)
  2. La porción de código a extraer puede modificar como máximo 1 variable temporal que se use luego. Ese será el valor de retorno del método extraído. Si hay más de una variable modificada no se puede extraer.

Refactoring: Extract Method

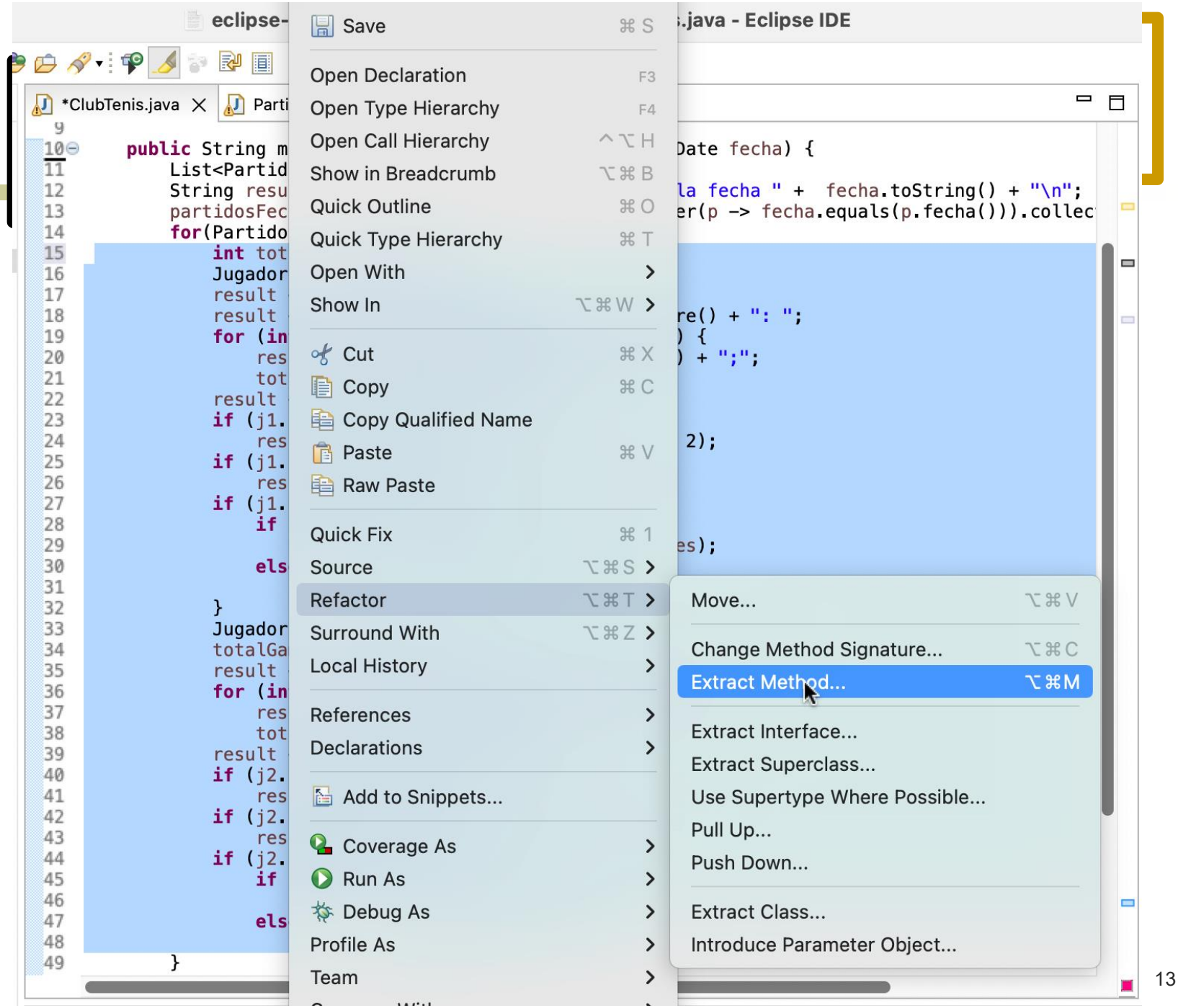
- **Mecánica**:
  1. Crear un nuevo método cuyo nombre explique su propósito
  2. Copiar el código a extraer al nuevo método
  3. Revisar las variables temporales del método original
     a) Si alguna se usa sólo en el código extraído, mover su declaración
     b) Si alguna se lee dentro del código extraído, pasar como parámetro
     c) Si hay una variable modificada por el código extraído que se usa luego en el método original, retornar el valor de esa variable
  4. Compilar
  5. Reemplazar el código extraído en el método original por una llamada al nuevo método. Si el nuevo retorna un valor, tratar como query y asignar
  6. Compilar y testear

Refactoring Extract method

```
public class ClubTenis {

 public String mostrarPuntajesJugadoresEnFecha(LocalDate
            fecha) {
   List<Partido> partidosFecha;
   String result = "Puntajes para los partidos de la fecha "
            +  fecha.toString() + "\n";
   partidosFecha = coleccionPartidos.stream().filter(p ->
            fecha.equals(p.fecha())).collect(Collectors.toList());
   for(Partido p: partidosFecha)
            result += this.mostrarPartido(p);
   return result;
 }

 private String mostrarPartido(Partido p) {...}
```


eclipse-
.java - Eclipse IDE
*ClubTenis.java
Parti
public String m
List<Partid
String resu
partidosFec
for(Partido
int tot
Jugador
result
result
for (in
res
tot
result
if (j1.
res
if (j1.
res
if (j1.
if
els
}
Jugador
totalGa
result
for (in
res
tot
result
if (j2.
res
if (j2.
res
if (j2.
if
els
}
Date fecha) {
la fecha " + fecha.toString() + "\n";
er(p -> fecha.equals(p.fecha())).collec
re() + ": ";
) {
) + ";";
2);
es);
Save ⌘S
Open Declaration F3
Open Type Hierarchy F4
Open Call Hierarchy ^⌥H
Show in Breadcrumb ⌥⌘B
Quick Outline ⌘O
Quick Type Hierarchy ⌘T
Open With
Show In ⌥⌘W
Cut ⌘X
Copy ⌘C
Copy Qualified Name
Paste ⌘V
Raw Paste
Quick Fix ⌘1
Source ⌥⌘S
Refactor ⌥⌘T
Surround With ⌥⌘Z
Local History
References
Declarations
Add to Snippets...
Coverage As
Run As
Debug As
Profile As
Team
Move... ⌥⌘V
Change Method Signature... ⌥⌘C
Extract Method... ⌥⌘M
Extract Interface...
Extract Superclass...
Use Supertype Where Possible...
Pull Up...
Push Down...
Extract Class...
Introduce Parameter Object...

Se requieren ajustes manuales

- Antes de poder aplicar “Extract Method” automáticamente:
  - El código que queremos extraer en un nuevo método necesita devolver 2 valores: result y totalGames
    - Entonces primero hay que mover la declaración de totalGames al código que vamos a extraer. Podemos hacerlo porque solo se usa en el código a extraer.
- Luego de aplicar “Extract Method”
  - Vemos que no hace falta pasar result como parámetro
    - Eliminamos el parámetro y volvemos a declarar result dentro del nuevo método.

A tener en cuenta…

- Testear siempre después de hacer un cambio
  - Sí se cometió un error es más fácil corregirlo
- Definir buenos nombres

En la clase ClubTenis…

- A quien pertenece realmente el codigo de

String mostrarPartido(Partido p) {…}. **?**

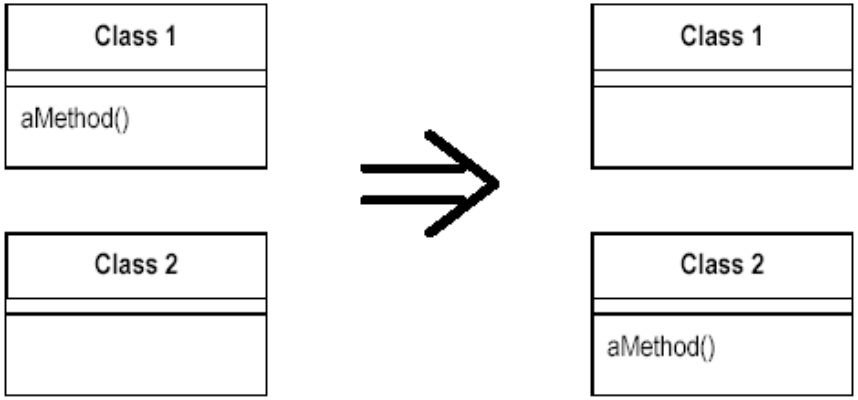
```
public class ClubTenis {

private String mostrarPartido(Partido partido) {
        int totalGames = 0;
        Jugador j1 = partido.jugador1();
        String result = “Partido: ” + “\n”;
        result += “Puntaje del jugador: “ + j1.nombre() + “: “;
        for (int gamesGanados: partido.puntosPorSetDe(j1)) {
                result += Integer.toString(gamesGanados) + “;”;
                totalGames += gamesGanados; }
        result += “Puntos del partido: “;
        if (j1.zona() == “A”) result += Integer.toString(totalGames * 2);
        if (j1.zona() == “B”) result += Integer.toString(totalGames);
        if (j1.zona() == “C”)
                if (partido.ganador() == j1)
                        result += Integer.toString(totalGames);
                else
                        result += Integer.toString(0);
        Jugador j2 = partido.jugador2();
…
...
        return result;
}
```

RESPONSABILIDAD MAL ASIGNADA O ENVIDIA DE ATRIBUTO

Refactoring “Move Method”

- Motivación:
  - Un método esta usando o usará muchos servicios que están definidos en una clase diferente a la suya
- Solucion:
  - Mover el método a la clase donde están los servicios que usa.
  - Convertir el método original en un simple delegación o eliminarlo



Refactoring Move Method

```
public class ClubTenis {
  public String mostrarPuntajesJugadoresEnFecha(LocalDate fecha) {
    List<Partido> partidosFecha;
    String result = "Puntajes para los partidos de la fecha " +  fecha.toString() + "\n";
    partidosFecha = coleccionPartidos.stream().filter(...));
    for(Partido p: partidosFecha)
            result = p.mostrar();
    return result;
  }

public class Partido {
  public String mostrar() {
            int totalGames = 0;
            Jugador j1 = jugador1();
            String result = "Partido: " + "\n";
            result += "Puntaje del jugador: " + j1.nombre() + ": ";
            ...
}
```

Move Method

- Precondiciones: para poder mover el método M1 de la clase A a la clase B tiene que ocurrir que:

  1. B no debe contener una definición del método M1, ni heredarla
  2. No debe haber otra definición de M1 en la superclase ni en subclases de A
  3. M1 no debe modificar variables de instancia de A
  4. M1 debe poder acceder desde B a todo lo que podía acceder desde A

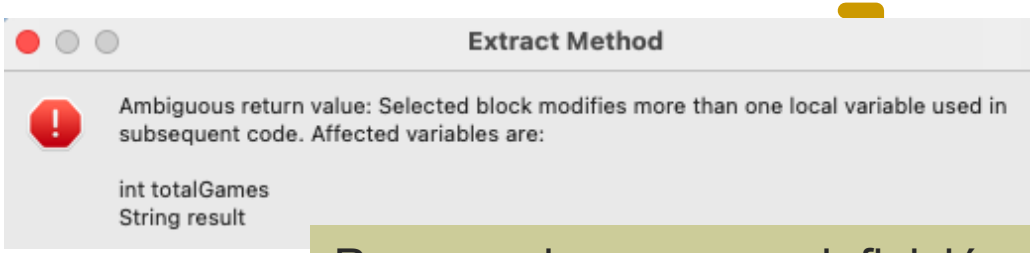
Move Method

- Mecánica:
  1. Revisar las v.i. usadas por el método a mover. Tiene sentido moverlas también?
  2. Declarar el método en la clase target (B). Considerar si debe cambiar de nombre
  3. Copiar el código a mover al nuevo método. Ajustar lo que haga falta
     - Qué se necesita pasar como parámetro?
     - Decidir qué clase va a manejar las excepciones
  4. Compilar la clase B
  5. Determinar como referenciar a B desde A
  6. Reemplazar el método original por una llamada al nuevo método en B
  7. Compilar y testear
  8. Decidir si borrar el método en A o mantenerlo como delegación

En la clase Partido…

- mostrar() ¿es un buen nombre?
- Aplicamos el refactoring Rename Method porque todo buen código debería comunicar con claridad lo que hace, sin necesidad de agregar comentarios. Lo llamamos toString()

- El metodo sigue siendo bastante largo, porque tiene código duplicado
  - más Extract Method!



Pero puedo mover su definición

```
public class Partido {
@Override
public String toString() {
        int totalGames = 0;
        Jugador j1 = jugador1();
        String result = “Partido: ” + “\n”;
        result += “Puntaje del jugador: “ + j1.nombre() + “: “;
        for (int gamesGanados: puntosPorSetDe(j1)) {
                result += Integer.toString(gamesGanados) + “;”;
                totalGames += gamesGanados; }
        result += “Puntos del partido: “;
        if (j1.zona() == “A”) result += Integer.toString(totalGames * 2);
        if (j1.zona() == “B”) result += Integer.toString(totalGames);
        if (j1.zona() == “C”)
                if (ganador() == j1)
                        result += Integer.toString(totalGames);
                else
                        result += Integer.toString(0);
        Jugador j2 = jugador2();
        totalGames = 0;
        result += “Puntaje del jugador: “ + j2.nombre() + “: “;
…       ...
```

Ajustando la declaración de temporales

```
public String puntosJugadorToString(Jugador unJugador) {
 int totalGames = 0;
 String result = "Puntaje del jugador: " + unJugador.nombre() +   ": ";
 for (int gamesGanados: puntosPorSetDe(unJugador)) {
            result += Integer.toString(gamesGanados) + ";";
            totalGames += gamesGanados; }
 result += "Puntos del partido: ";
 if (unJugador.zona() == "A")
            result += Integer.toString(totalGames * 2);
 if (unJugador.zona() == "B")
            result += Integer.toString(totalGames);
 if (unJugador.zona() == "C")
            if (this.ganador() == unJugador)
              result += Integer.toString(totalGames);
            else
              result += Integer.toString(0);
 return result;
}
```

Luego de ajustar en el método original

```
@Override
public String toString() {
  Jugador j1 = jugador1();
  String result = "Partido: " + "\n";
  result += puntosJugadorToString(j1);
  Jugador j2 = jugador2();
  result += puntosJugadorToString(j2);
  return result;
}
```

Seguimos revisando

- puntosJugadorToString() ¿está bien ubicado en Partido?
  - No!
- Aplico **Move Method**
  Partido>>puntosJugadorToString(Jugador j) a
  Jugador>>puntosEnPartidoToString(Partido p)

- El refactoring Move Method es uno de los más utilizados. Permite ir repartiendo las responsabilidades adecuadamente entre las clases, resolviendo las instancias de Feature Envy

Move Method

New target for 'String puntosJugadorToString(Jugador unJugador)':

Type	Receiver
Jugador	unJugador
Jugador	ganador

New method name: puntosEnPartido

New parameter name: partido

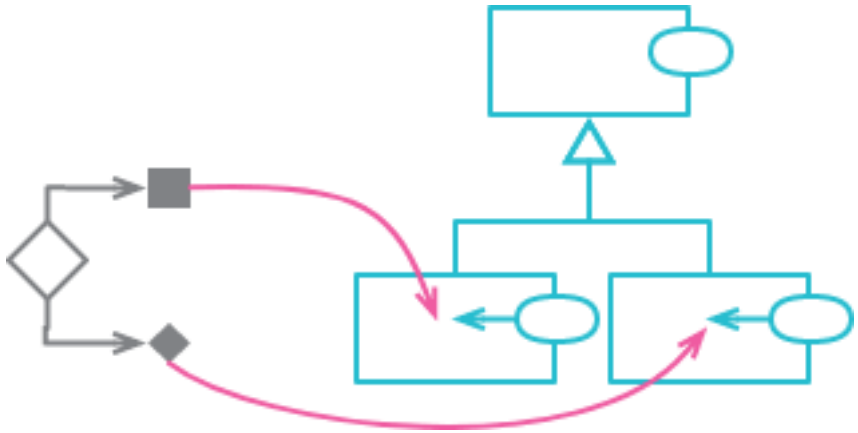
☐ Keep original method as delegate to moved method

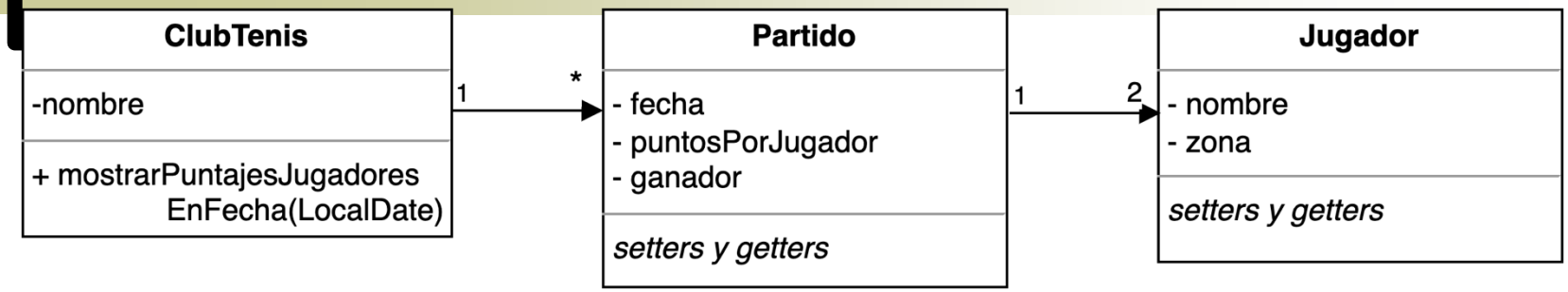
☑ Mark as deprecated

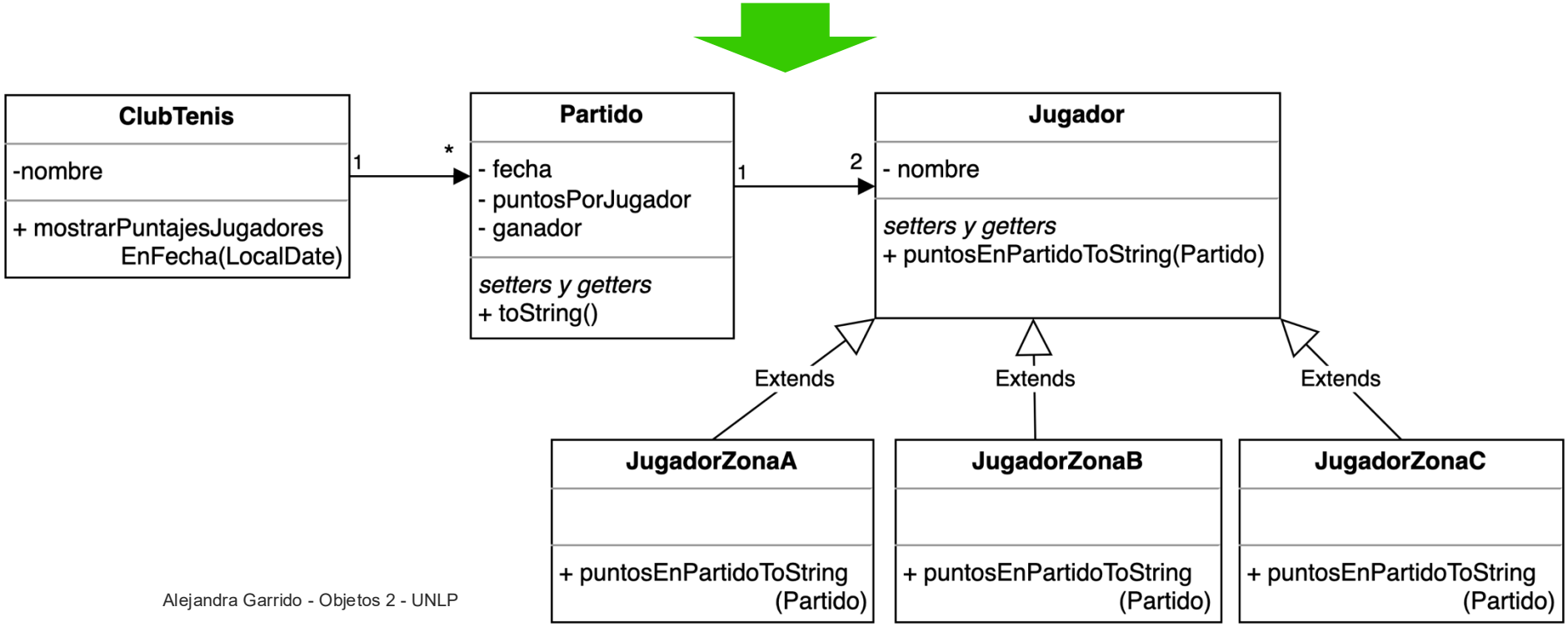
Preview > | Cancel | OK

Seguimos teniendo el switch

- ¿Cómo eliminar sentencias condicionales?
- ➔ Refactoring: Replace Conditional with Polymorphism




ClubTenis
-nombre
+ mostrarPuntajesJugadores EnFecha(LocalDate)
1
*
Partido
- fecha
- puntosPorJugador
- ganador
setters y getters
1
2
Jugador
- nombre
- zona
setters y getters


ClubTenis
-nombre
+ mostrarPuntajesJugadores EnFecha(LocalDate)
1
*
Partido
- fecha
- puntosPorJugador
- ganador
setters y getters
+ toString()
1
2
Jugador
- nombre
setters y getters
+ puntosEnPartidoToString(Partido)
Extends
Extends
Extends
JugadorZonaA
+ puntosEnPartidoToString (Partido)
JugadorZonaB
+ puntosEnPartidoToString (Partido)
JugadorZonaC
+ puntosEnPartidoToString (Partido)

Replace Conditional with Polymorphism

- Mecánica:
  1. Crear la jerarquía de clases necesaria
  2. Si el condicional es parte de un método largo: Extract Method
  3. Por cada subclase:
     1. Crear un método que sobreescribe al método que contiene el condicional
     2. Copiar el código de la condición correspondiente en el método de la subclase y ajustar
     3. Compilar y testear
     4. Borrar la condición y código del branch del método en la superclase
     5. Compilar y testear
  4. Hacer que el método en la superclase sea abstracto

Antes de Replace Conditional with Polymorphism

Jugador>>puntosEnPartidoToString(Partido partido)

```
…
  if (zona() == "A")
            result += Integer.toString(totalGames * 2);
  if (zona() == "B")
            result += Integer.toString(totalGames);
  if (zona() == "C")
            if (partido.ganador() == this)
              result += Integer.toString(totalGames);
            else
              result += Integer.toString(0);
```

Después de Replace Conditional y ajustes

```
public class Jugador {
 public String puntosEnPartidoToString(Partido partido) {
  int totalGames = 0;
  String result = "Puntaje del jugador: " + nombre() + ": ";
  for (int gamesGanados: partido.puntosPorSetDe(this)) {
            result += Integer.toString(gamesGanados) + ";";
            totalGames += gamesGanados; }
  result += "Puntos del partido: ";
  result +=
    Integer.toString(this.puntosGanadosEnPartido(partido,
                                                                    totalGames));
  return result;
}
public class JugadorZonaA {
 public int puntosGanadosEnPartido(Partido partido, int totalGames) {
  return totalGames * 2;
 }
```

Ajustando (2)

```
public class Jugador {
 public String puntosEnPartidoToString(Partido partido) {
  int totalGames = 0;
  String result = "Puntaje del jugador: " + nombre() + ": ";
  for (int gamesGanados: partido.puntosPorSetDe(this)) {
            result += Integer.toString(gamesGanados) + ";";
            totalGames += gamesGanados; }
  result += "Puntos del partido: ";
  result +=
   Integer.toString(this.puntosGanadosEnPartido(partido,
                                                                    totalGames));
  return result;
}
public class JugadorZonaA {
 public int puntosGanadosEnPartido(Partido partido, int totalGames) {
  return totalGames * 2;
 }
```

Refactoring: Replace Temp with Query

- Motivación: usar este refactoring:
  - Para evitar métodos largos. Las temporales, al ser locales, fomentan métodos largos
  - *Para poder usar una expresión desde otros métodos*
  - Antes de un Extract Method, para evitar parámetros innecesarios
- Solución:
  - Extraer la expresión en un método
  - Remplazar TODAS las referencias a la var. temporal por la expresión
  - El nuevo método luego puede ser usado en otros métodos

Ajustando(3)

```
public class Jugador {
 public String puntosEnPartidoToString(Partido partido) {
  String result = "Puntaje del jugador: " + nombre() + ": ";
  for (int gamesGanados: partido.puntosPorSetDe(this))
            result += Integer.toString(gamesGanados) + ";";
  result += "Puntos del partido: ";
  result +=
            Integer.toString(this.puntosGanadosEnPartido(partido));
  return result;   }

 public int totalGamesEnPartido(Partido partido) {
  int totalGames;
  for (int gamesGanados: partido.puntosPorSetDe(this))
            totalGames += gamesGanados;
  return totalGames;
 }

public class JugadorZonaA {
 public int puntosGanadosEnPartido(Partido partido) {
  return totalGamesEnPartido(partido) * 2;
 }
```

Sobre la performance

- La mejor manera de optimizar un programa, primero es escribir un programa bien factorizado y luego optimizarlo
- En el ejemplo, mientras estamos refactorizando no importa que el cambio nos haga recorrer más de una vez la colección de sets de un partido, no solo porque la colección es corta sino que además ganamos en flexibilidad y reusabilidad
  - El código es más flexible porque el día de mañana podemos cambiar la manera de generar los datos que se muestran sin que eso cambie la manera de calcular los puntos; ambos algoritmos pueden cambiar de manera independiente
  - El código es más reusable porque ahora cuento con un método que calcula los puntos de cada jugador y lo puedo invocar también en otros casos donde no necesite devolver un string con los resultados